

# Quantization: Memory & Quality.

A trained weight is a real number, but it need not be *stored* as one. Quantization maps each weight to a low-bit integer through an affine rule, slashing memory in direct proportion to the bit width: FP16 to INT8 halves it, INT4 cuts three-quarters. The price is precision — the dequantized weight differs from the original, and that error grows as bits shrink. We derive the affine quantize/dequantize formulas, quantize a concrete vector to INT8 and INT4 by hand, and measure the error that makes INT4 risky. Every number below is computed in full — nothing summarised or deferred.

## THE EQUATION

$$q = \text{round}\left(\frac{x}{s}\right) + z, \quad \hat{x} = s(q - z), \quad s = \frac{x_{\max} - x_{\min}}{2^b - 1}$$

**Worked example.** A 7B model:  $\text{FP16} = 7 \times 10^9 \cdot 2 = 14 \text{ GB}$ ,  $\text{INT8} = 7 \text{ GB}$ ,  $\text{INT4} = 3.5 \text{ GB}$ . Quantizing the vector  $[-0.8, -0.1, 0, 0.3, 1.2, 2.5]$ : max error 0.0035 at INT8 vs 0.10 at INT4 — about 28× worse.

Topic 2 of 3 in this module · companion to the course page · v1.0

## Contents

---

|   |                                                    |   |
|---|----------------------------------------------------|---|
| 1 | The claim: bits buy memory, cost precision         | 2 |
| 2 | A diagram: the integer grid                        | 3 |
| 3 | The derivation: memory scales linearly with bits   | 3 |
| 4 | Worked example: quantize a vector to INT8 and INT4 | 3 |
| 5 | Speed, and why the gain is real                    | 4 |
| 6 | Properties / consequences that matter              | 5 |
| 7 | Where this sits in the track                       | 5 |
| A | Reproducible (NumPy)                               | 5 |

### 1 The claim: bits buy memory, cost precision

Store each weight in  $b$  bits instead of 16 and memory scales by  $b/16$ . The mapping is *affine*: a real value  $x$  becomes an integer  $q$  via a scale  $s$  and an integer zero-point  $z$ , and is recovered (approximately) by dequantizing.

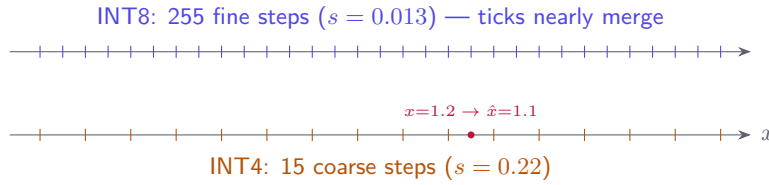
$$\boxed{q = \text{round}\left(\frac{x}{s}\right) + z, \quad \hat{x} = s(q - z), \quad s = \frac{x_{\max} - x_{\min}}{2^b - 1}, \quad z = \text{round}\left(-\frac{x_{\min}}{s}\right)} \quad (1)$$

The scale  $s$  is the width of one integer step; the zero-point  $z$  is the integer that maps back to real 0, which lets an asymmetric range ( $x_{\min} \neq -x_{\max}$ ) use the full integer codebook. Dequantized  $\hat{x}$  lands on the nearest grid point to  $x$ , so the round-off error is bounded by half a step,  $|x - \hat{x}| \leq s/2$  (before clipping).

| Symbol    | Meaning                     | Value / shape       |
|-----------|-----------------------------|---------------------|
| $b$       | bits per stored value       | 16, 8, 4            |
| $2^b - 1$ | integer codebook span       | 65535, 255, 15      |
| $s$       | scale (real value per step) | derived per Eq. (1) |
| $z$       | integer zero-point          | derived per Eq. (1) |
| $x$       | original real weight        | fp32/fp16           |
| $\hat{x}$ | dequantized weight          | $s(q - z)$          |
| $P$       | model parameters            | $7 \times 10^9$     |

Quantization is rounding to a ruler with coarse ticks. The scale  $s$  is the spacing between ticks: a fixed real range  $[x_{\min}, x_{\max}]$  is divided into  $2^b - 1$  equal steps, so doubling the bits quadruples the number of ticks and quarters the spacing. INT8's 255 ticks sit close together (error tiny); INT4's 15 ticks are coarse (error large). You are trading the smoothness of the weight grid for the bytes it takes to store an index into it.

## 2 A diagram: the integer grid



A real value snaps to the nearest tick. With INT8 the ticks are so dense the snap is invisible; with INT4 a value like 1.2 snaps to 1.1 — a visible 0.1 error.

## 3 The derivation: memory scales linearly with bits

Bytes per parameter is  $b/8$ , so total memory is  $P \cdot b/8$ . For  $P = 7 \times 10^9$ :

**FP16:**  $7 \times 10^9 \cdot 2 = 14 \times 10^9$  B = 14 GB,    **INT8:**  $7 \times 10^9 \cdot 1 = 7$  GB,    **INT4:**  $7 \times 10^9 \cdot 0.5 = 3.5$  GB.

INT8 is a 50% cut, INT4 a 75% cut, relative to FP16:

$$\text{mem}(b) = P \cdot \frac{b}{8}, \quad \frac{\text{mem}(b)}{\text{mem}(16)} = \frac{b}{16} \quad (2)$$

## 4 Worked example: quantize a vector to INT8 and INT4

Take the weight vector  $x = [-0.8, -0.1, 0, 0.3, 1.2, 2.5]$ , so  $x_{\min} = -0.8$ ,  $x_{\max} = 2.5$ , range = 3.3.

**INT8** ( $2^8 - 1 = 255$ ):  $s = 3.3/255 = 0.0129412$ ,  $z = \text{round}(0.8/s) = 62$ . Then  $q = \text{round}(x/s) + z$  gives

$$q = [0, 54, 62, 85, 155, 255], \quad \hat{x} = s(q - z) = [-0.8024, -0.1035, 0, 0.2976, 1.2035, 2.4976].$$

Element-wise error  $|x - \hat{x}| = [0.0024, 0.0035, 0, 0.0024, 0.0035, 0.0024]$ , so  $\max |x - \hat{x}| = 0.0035$ .

**INT4** ( $2^4 - 1 = 15$ ):  $s = 3.3/15 = 0.2200$ ,  $z = \text{round}(0.8/0.22) = 4$ . Then

$$q = [0, 4, 4, 5, 9, 15], \quad \hat{x} = s(q - z) = [-0.88, 0, 0, 0.22, 1.10, 2.42].$$

Element-wise error  $|x - \hat{x}| = [0.08, 0.10, 0, 0.08, 0.10, 0.08]$ , so  $\max |x - \hat{x}| = 0.10$ . INT4's error is  $0.10/0.0035 \approx 28\times$  that of INT8 — exactly the coarser grid ( $s$  is  $17\times$  larger, and rounding to fewer levels compounds it). The combined picture:

| Precision | bytes/param | 7B memory | scale $s$ | max error | typical quality drop     |
|-----------|-------------|-----------|-----------|-----------|--------------------------|
| FP16      | 2           | 14.0 GB   | —         | 0         | baseline                 |
| INT8      | 1           | 7.0 GB    | 0.01294   | 0.0035    | $\sim 0.1\text{--}0.3\%$ |
| INT4      | 0.5         | 3.5 GB    | 0.22000   | 0.1000    | $\sim 0.5\text{--}1\%$   |

The heatmap shades the max quantization error (intensity  $\propto$  error, capped at **rose!80** for the largest):

|           | FP16    | INT8   | INT4   |
|-----------|---------|--------|--------|
| max error | 0.0000  | 0.0035 | 0.1000 |
| 7B memory | 14.0 GB | 7.0 GB | 3.5 GB |

(Rose shading is linear in error relative to INT4's 0.10; indigo is linear in memory relative to FP16's 14 GB.) Two trends collide: memory shrinks linearly with bits, but error climbs sharply — hence the comfort with INT8 and the care required for INT4.

Memory falls *linearly* in bits, but reconstruction error rises *super-linearly* as the grid coarsens. INT8 sits in a sweet spot — 50% memory for  $\sim 0.1\%$  quality loss — while INT4 (75% memory) demands smarter methods (GPTQ, AWQ) to keep the loss near 0.5–1% rather than the multi-percent drop that naive round-to-nearest can incur.

## 5 Speed, and why the gain is real

LLM decoding is memory-bandwidth-bound: each step *reads* the entire weight matrix to produce one token (Topic 3). Halving the bytes per weight roughly halves the bytes moved, so INT8/INT4 decoding runs about  $1.5\text{--}2\times$  faster even though the arithmetic is unchanged — the bottleneck was never the math, it was fetching weights from HBM. Smarter post-training methods improve *quality* at a given bit width:

- **GPTQ** minimizes the layer-wise output error (not the weight error), reordering and compensating columns so the cumulative effect on activations is small.

- **AWQ** (activation-aware) scales up the channels that matter most for the output before rounding, protecting salient weights from the coarse grid.

Both beat naive round-to-nearest precisely because the metric that matters is the model's *output*, not the raw weight values.

## 6 Properties / consequences that matter

1. **Linear memory law.**  $\text{mem} = P \cdot b/8$ : every halving of bits halves storage.
2. **Error bounded by  $s/2$ .** Smaller range or more bits both shrink  $s$  and the error; clipping outliers shrinks the range at the cost of mangling the clipped values.
3. **Bandwidth, not FLOPs, is the win.** The decode speedup tracks bytes moved, so it shows up most in the memory-bound regime (small batch, long context).
4. **Granularity matters.** Per-tensor scales are cheapest; per-channel or per-group scales cost a little metadata but greatly reduce error when ranges vary across channels.

**Outlier activations wreck naive quantization.** A handful of channels in LLM activations carry values orders of magnitude larger than the rest. A single per-tensor scale must stretch to cover those outliers, which blows up  $s$  and rounds every *ordinary* value to a coarse, error-laden grid. The fix is to not use one scale for everything: per-channel (per-group) scales, or mixed precision that keeps the outlier channels in higher precision (the LLM.int8() idea), or AWQ-style salience-aware scaling. Quantizing weights and activations together without handling outliers is the classic way to turn a 0.3% drop into a double-digit collapse.

## 7 Where this sits in the track

Module 2.4 counted parameters and inference FLOPs; this topic shrinks the *bytes* those parameters occupy. It compounds with Topic 1 (PagedAttention frees cache memory) and Topic 3 (quantized weights move fewer bytes per decode step, raising throughput at the bandwidth-bound knee). Together the three define the production serving stack: shrink the weights, parcel the cache efficiently, and batch in flight.

### A Reproducible (NumPy)

Inputs:  $x = [-0.8, -0.1, 0, 0.3, 1.2, 2.5]$ ,  $P = 7 \times 10^9$ . The snippet reproduces the memory sizes (14/7/3.5 GB), the scales (0.01294, 0.220), the integer codes, and the max errors (0.0035, 0.10).

```
import numpy as np
P = 7e9
for bits in (16,8,4):
    print(bits, "bits ->", P*(bits/8)/1e9, "GB")    # 14.0, 7.0, 3.5

x = np.array([-0.8,-0.1,0.0,0.3,1.2,2.5])
def aq(x, bits):
    qmax = 2**bits - 1
```

```
s = (x.max()-x.min())/qmax
z = round(-x.min()/s)
q = np.clip(np.round(x/s)+z, 0, qmax)
xhat = s*(q-z)
return s, z, q.astype(int), xhat, np.abs(x-xhat).max()
for bits in (8,4):
    s,z,q,xhat,e = aq(x,bits)
    print("INT%d s=%.5f z=%d maxerr=%.4f"%(bits,s,z,e))
# INT8 s=0.01294 z=62 maxerr=0.0035
# INT4 s=0.22000 z=4 maxerr=0.1000
```

---

Marevlo Research · Transformer Track · Module 7.3 · Topic 2 of 3.